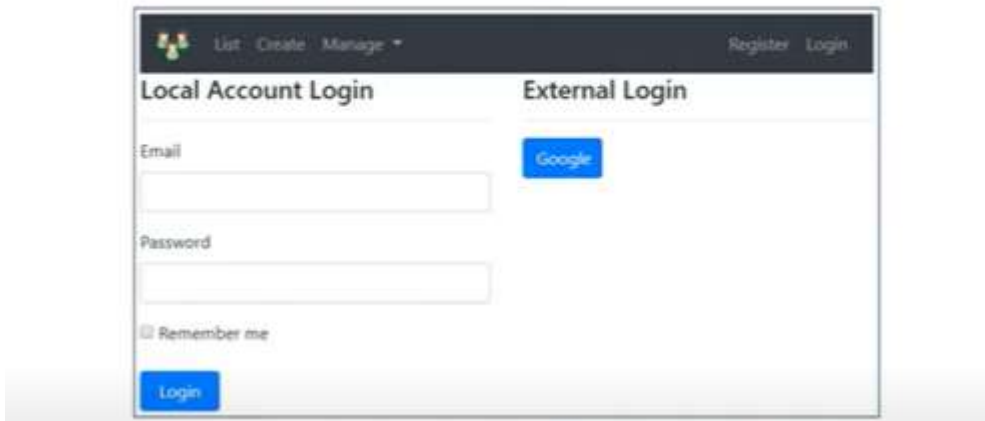
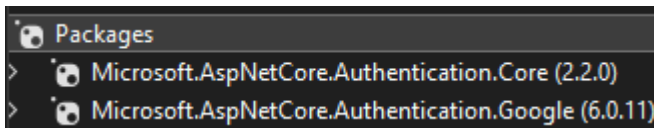


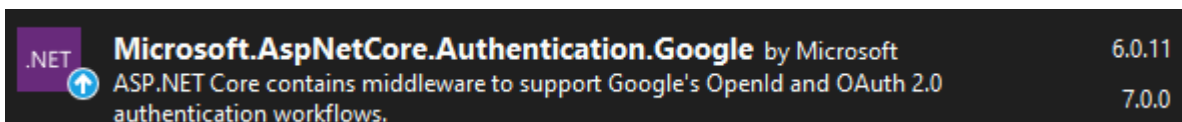
Google Authentication - Setting up the UI



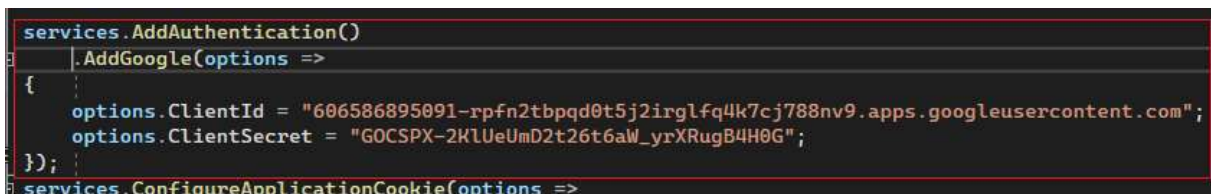
Трябва да инсталираме [Microsoft.AspNetCore.Authentication.Google](#) NuGet package, за да бъде достъпен метод [AddGoogle\(\)](#).



Съответно аналогичен е пакетът за Facebook, Twitter, Microsoft. Най-добре да се инсталира направо [Microsoft.AspNetCore.Authentication.Core](#), която след версия 2.2 би трябвало да ги съдържа, но практически не ми разпозна метод `.AddGoogle` докато не инсталирах по-старата версия `.Google 6.0`, защото последната 7.0 не ми разрешава.



<https://learn.microsoft.com/en-us/aspnet/core/security/authentication/social/?view=aspnetcore-7.0&tabs=visual-studio>



```
services.AddAuthentication()  
    .AddGoogle(options =>  
    {  
        options.ClientId = "606586895091-  
rpfnt2bpqd0t5j2irglfq4k7cj788nv9.apps.googleusercontent.com";  
        options.ClientSecret = "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";  
    });  
services.ConfigureApplicationCookie(options =>
```

Credintials-> Edit ->

Name	Creation date	Type	Client ID	Actions
Employee Mgmt Client	Nov 24, 2022	Web application	606586895091-rpfn...	  

Name *

Employee Mgmt Client

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

Client ID

606586895091-rpfn2tbpqd0t5j2rglfq4k7c798nv9.aps.googleusercontent.com

Client secret

GOCSPX-2KJUEImD2t26t6aW_yXRugB4H0G

Creation date

November 24, 2022 at 6:13:17 PM GMT+2

The domains of the URIs you add below will be automatically added to your OAuth consent screen as authorized domains.

Следваща стъпка е да добавим възможност за външно вписване в изгледа и за целта трябва да добавим нови свойства в модела:

```

LoginViewModel.cs | Startup.cs | SuperAdminHandler.cs
EmployeeManagement.ViewModels.LoginViewModel | ExternalLogins
4 references
public bool RememberMe { get; set; }

0 references
public string returnUrl { get; set; }

0 references
public IList<AuthenticationScheme> ExternalLogins { get; set; }

```

```
public string returnUrl { get; set; }
```

```
public IList<AuthenticationScheme> ExternalLogins { get; set; }
```

ReturnUrl съхранява адреса, за да може да се върне потребителят на него.

ReturnUrl is the URL the user was trying to access before authentication. We preserve and pass it between requests using **ReturnUrl** property, so the user can be redirected to that URL upon successful authentication.

ExternalLogins property stores the list of external logins (like Facebook, Google etc) that are enabled in our application.

AuthenticationScheme is in Microsoft.AspNetCore.Authentication namespace

Следва да попълним тези 2 нови properties преди да предоставим този LoginViewModel.

AccountController -> Login action.

```

AccountController.cs* | CanEditOnlyOt...imsHa
EmployeeManagement
[HttpGet]
[AllowAnonymous]
0 references
public IActionResult Login()
{
    return View();
}

```

Подаваме променлива returnUrl:

```

EmployeeManagement.Controllers.AccountController Login(string returnUrl)
[HttpGet]
[AllowAnonymous]
0 references
public async Task<IActionResult> Login(string returnUrl)
{
    LoginViewModel model = new LoginViewModel
    {
        ReturnUrl = returnUrl,
        ExternalLogins =
            (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList();
    };

    return View(model);
}

```

model binding

instance

Google Authentication - Setting up the UI

Views
Account
Login.cshtml
Register.cshtml

List Create Manage
Register Login

Local Account Login

External Login

Email

Password

☐ Remember me

Искаме да разделим страницата на 2, съответно намаляваме `<div class="col-md-12">` на 6.

Демонстративно:

```
AccountController.cs Login.cshtml* CanEditOnlyOt...imsHandler.cs
@model LoginViewModel

@{
    ViewBag.Title = "User Login";
}

<div class="row">
    <div class="col-md-6">_</div>

    <div class="col-md-6">
        <h1>External Login</h1>
        <hr />
        @{
            if (Model.ExternalLogins.Count == 0)
            {
                <div>No external login configured</div>
            }
        }
    </div>
</div>
```

Закоментираме:

```
EmployeeManagement.Startup
//services.AddAuthentication()
// .AddGoogle(options =>
//{
//    options.ClientId = "606586895091-rpfm2tbpqd0t5j2irg
//    options.ClientSecret = "GOCSPX-2KlUeUmD2t26t6aW_yrXf
//});
```

Browser address bar: <https://localhost:44346/account/login>

List Create

Local Account Login

Email

Password

☐ Remember me

Login

External Login

No external login configured

Разкоментираме

```

Login.cshtml* Startup.cs* LoginViewModel.cs
<div class="col-md-6">
  <h1>External Login</h1>
  <hr />
  @{
    if (Model.ExternalLogins.Count == 0)
    {
      <div>No external login configured</div>
    }
    else
    {
      <form method="post" asp-action="ExternalLogin" asp-route-returnUrl="@Model.ReturnUrl">
        <div>
          @foreach (var provider in Model.ExternalLogins)
          {
            <button type="submit" class="btn btn-primary"
              name="provider" value="@provider.Name"
              title="Login using your @provider.DisplayName account">
              @provider.DisplayName
            </button>
          }
        </div>
      </form>
    }
  }
</div>

```

```

<div class="col-md-6">
  <h1>External Login</h1>
  <hr />
  @{
    if (Model.ExternalLogins.Count == 0)
    {
      <div>No external login configured</div>
    }
    else
    {
      <form method="post" asp-action="ExternalLogin" asp-route-
returnUrl="@Model.ReturnUrl">
        <div>
          @foreach (var provider in Model.ExternalLogins)
          {
            <button type="submit" class="btn btn-primary"
              name="provider" value="@provider.Name"
              title="Login using your @provider.DisplayName
account">
              @provider.DisplayName
            </button>
          }
        </div>
      </form>
    }
  }
</div>

```

External Login

Google

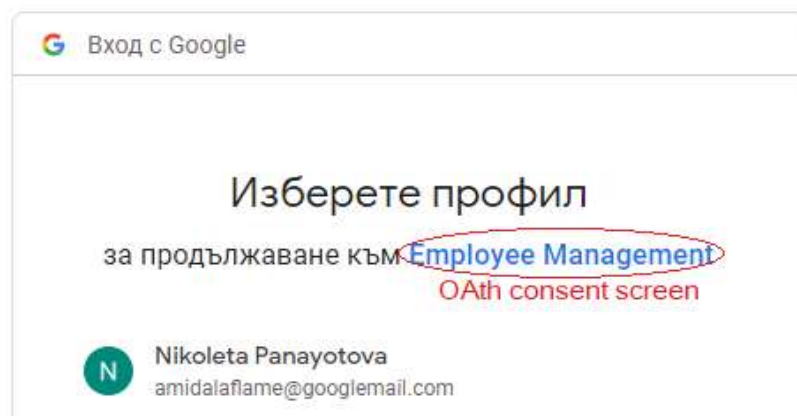
Login using your Google account

+ Tool Tip -> <https://localhost:44346/account/ExternalLogin>

Остава ExternalLogin action -> AccountController

```
AccountController.cs LoginViewModel.cs
EmployeeManagement.Controllers.AccountController ExternalLogin(string provider, string returnUrl)
[AllowAnonymous]
[HttpPost]
0 references
public IActionResult ExternalLogin(string provider, string returnUrl)
{
    <button name="provider" MVC auto binding to do
    var redirectUrl = Url.Action("ExternalLoginCallback", "Account",
        new { ReturnUrl = returnUrl });
    var properties = signInManager
        .ConfigureExternalAuthenticationProperties(provider, redirectUrl);
    return new ChallengeResult(provider, properties);
}
```

accounts.google.com/o/oauth2/v2/auth/oauthchooseaccount?response_type=code&client_id=60658689



<https://localhost:44346/Account/ExternalLoginCallback> -> 404

В следващия урок ще видим имплементирането му.

Configure Google Authentication

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddAuthentication().AddGoogle(options =>
    {
        options.ClientId = "XXXXX";
        options.ClientSecret = "YYYYY";
    });
}
```

LoginViewModel Changes

```
public class LoginViewModel
{
    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Required]
    [DataType(DataType.Password)]
    public string Password { get; set; }

    [Display(Name = "Remember me")]
    public bool RememberMe { get; set; }

    public string returnUrl { get; set; }

    public IList<AuthenticationScheme> ExternalLogins { get; set; }
}
```

Login Action in AccountController

```
[HttpGet]
[AllowAnonymous]
public async Task<IActionResult> Login(string returnUrl)
{
    LoginViewModel model = new LoginViewModel
    {
        returnUrl = returnUrl,
        ExternalLogins =
            (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList()
    };

    return View(model);
}
```

Login View

```
<div class="col-md-6">
    <h1>External Login</h1><hr />
    @{
        if (Model.ExternalLogins.Count == 0)
        {
            <div>No external logins configured</div>
        }
        else
        {
            <form method="post" asp-action="ExternalLogin" asp-route-returnUrl="@Model.ReturnUrl">
                <div>
                    @foreach (var provider in Model.ExternalLogins)
                    {
                        <button type="submit" class="btn btn-primary"
                            name="provider" value="@provider.Name"
                            title="log in using your @provider.DisplayName account">
                            @provider.DisplayName
                        </button>
                    }
                </div>
            </form>
        }
    }
</div>
```

ExternalLogin action - AccountController

```
[AllowAnonymous]
[HttpPost]
public IActionResult ExternalLogin(string provider, string returnUrl)
{
    var redirectUrl = Url.Action("ExternalLoginCallback", "Account",
        new { ReturnUrl = returnUrl });
    var properties = signInManager
        .ConfigureExternalAuthenticationProperties(provider, redirectUrl);
    return new ChallengeResult(provider, properties);
}
```